Optimizing Code-Mixed Language Identification with Adam and Gradient-Based Deep Neural Networks

Kanika Debbarma[1], Enjula Uchoi[1], Koj Sambyo[2], Premananda Sahu[3], Devendra Kumar Yadav[4*]

[1]Department of Computer Science & Engineering, National Institute of Technology , Jote, 791123, Arunachal Pradesh, India.
[2]Department of Computer Science & Engineering, National Institute of Technology , Jote, 791123, Arunachal Pradesh, India.
[3]School of Computer Science & Engineering, Lovely Professional University , Jalandhar, 144401, punjab, India.
[4*]School of Computer Science & Engineering, XIM University , Bhubaneswar, 752050, Odisha, India.

*Corresponding author(s). E-mail(s): devendra@xim.edu.in;
Contributing authors: kanika.phd26@nitap.ac.in; enjula.phd23@nitap.ac.in; kojsambyo@nitap.ac.in; prema.uce@gmail.com;

Abstract

Due to the rapid growth of multilingualism in social media platforms, code-mixed language identification critical task in Natural Language processing challenges. This research shows an outstanding approach to enhance code-mixed language identification and to overcome this critical task. The existing monolingual models achieve $>$ 95% accuracy on clean text, and due to lexical ambiguity and gradient sparsity, their performance drops to 60-80% on code-mixed data. We The proposed CNN-BiLSTM-Adam architecture combines convolutional feature extraction with bidirectional sequential context modeling for token-level language identification in code-mixed text. The contribution lies in the integration of adaptive gradient optimization, evaluation across diverse code-mixed language pairs, and analysis of low-resource Kokborok language identification. . The Adam's adaptive learning $\boldsymbol{\beta_1 = 0.9 and \beta_2 = 0.999}$ handles the sparse gradients in code-mixed text. We have trained on five languages code-mixed pairs,

including low-resource languages of Kokborok- Bengali with a variety of datasets available over the internet on different social platforms. Our model achieves 92.6% significance in accuracy, 92.6% precision, 92.4% recall and 92.5% F1-score- The performance improvements indicate that the proposed CNN–BiLSTM–Adam architecture effectively captures both local lexical patterns and long-range contextual dependencies within multilingual code-mixed text. The integration of Adam optimization further contributes to stable convergence and efficient learning, resulting in improved token-level language identification performance across multiple language pairs..

Keywords: Code-mixed language identification, Adam optimization, deep neural networks, gradient-based learning, multilingual NLP

1 Introduction

The growth of social platforms has led to the universal use of mixed language, where an individual mixes multiple languages, which poses significant obstacles for natural language processing (NLP), especially in language identification. Traditional models like monolingual models struggle to identify and process code-mixed text due to its combined nature. This paper put forward the problem of code-mixed language identification (CM-LID) by expressing a deep-learning-based approach optimized using Adam and gradient-based techniques. The core purpose of this study is to engineer a reliable and expandable model for CM-LID that can navigate the complexity of code-mixed text. We can implement deep neural networks (DNNs) with sophisticated optimization techniques to improve the accuracy and productivity of language identification. The following model is tested on a dataset consisting of code-mixed text from social media platforms, and the results are compared with state-of-the-art methods. The results showcase the dominance of the proposed approach, emphasizing its potential for practical applications in multilingual societies.

Considering the Hindi-English tweet dataset of the social media. We understood that the traditional tools used for language identification misclassify it as monolingual, which loses the crucial sentiment of understanding the cues. Our work specifies those challenges and covers three research gaps.

i. There is no existing model which has optimized for gradient sparsity in code-mixing and code-switching ii. Limited research and evaluation on low-resource language pairs like Kokborok-English and Kokborok-Bengali pairs iii. Despite their high compute costs, there is an over-reliance on transformers.

1.1 Research Contributions

Although the CNN–BiLSTM architecture itself is not entirely novel, the novelty of this work lies in its gradient-aware optimization strategy, extensive evaluation across five code-mixed language pairs, and investigation of low-resource Kokborok language identification. The major contributions of this work are:

1. Development of a gradient-optimized CNN–BiLSTM framework for token-level language identification in code-mixed text.

2. Evaluation across five code-mixed language pairs including low-resource Kokborok-English and Kokborok-Bengali datasets.

3. Investigation of the effect of adaptive gradient optimization on multilingual language identification performance.

4. Comprehensive comparison with deep learning and transformer-based baselines.

5. Analysis of code-switching patterns and model behavior through ablation and error analysis.
The objective of this study is to perform token-level language identification in code-mixed text. Given a code-mixed input sequence containing tokens from multiple languages, the proposed framework predicts a language label for each token in the sequence. The task is formulated as a token-level sequence labeling problem.

2 Literature Review

A hierarchical transformer framework for code-mixed texts, combining multi-headed self-attention and outer product attention, achieves significant performance gains across multiple languages and NLP tasks[1]. Multichannel neural networks (CNN + LSTM) and Bi-LSTM-CRF achieve high accuracy (93%) for word-level language identification in code-mixed data, while Transformer models excel in Kannada-English and Hindi-English contexts[2].MAML and ProtoMAML improve offensive language detection in Tamil-English and Malayalam-English code-mixed texts, especially in low-resource settings, outperforming traditional models in hate speech and gendered abuse detection tasks[3].

Techniques like CNN-GCBGA for teaching evaluation, quantum LSTM for social media language identification, and feature fusion with HAN for sentiment analysis in Malayalam demonstrate state-of-the-art performance in handling code-mixed and multilingual data [4]. State-of-the-art techniques improve Named Entity Recognition (NER) for Arabic-English code-switched data using contextual embeddings and data augmentation, addressing gaps in annotated corpora [5].Deep learning and machine learning methods outperform keyword matching in identifying vulgar language, with logistic regression achieving 0.91 accuracy, highlighting the limitations of keyword-based approaches [6]. Convolutional Neural Networks (CNNs) achieve high accuracy (0.9828) in emotion analysis for Mexican Spanish, with potential for further improvement through refined architectures [7]. Meta-learning methods outperform traditional models in detecting hate speech in Hindi-English code-mixed tweets, achieving high F1 scores and demonstrating the effectiveness of transfer learning [8]. A shared task at ICON 2023 focused on detecting gendered abuse in Hindi, Tamil, and Indian English, with F1 scores up to 0.616, highlighting challenges in abusive content identification

[9]. Ensemble systems achieve 92.6% accuracy for word-level language tagging in code-mixed data using minimal resources, demonstrating effective strategies for low-resource settings[10] A mix of CNN and bi-RNN improves text classification by avoiding unregistered word issues and capturing contextual information, outperforming traditional CNN and LSTM models [11]. A multilingual (Hindi-English) multimodal model (MHaDiG) generates sentiment, humor, and emotion-aware dialogues, showing the benefits of multilingualism in multimodal conversational settings [12]. A systematic review identifies challenges like ambiguity and lexical borrowing in code-mixed language identification (LID) and proposes a conceptual framework for future research [13]. Quantum LSTM model accurately predicts languages in Hindi-English code-mixed social media texts, paving the way for quantum-inspired machine learning applications [14]. Word embeddings (CBOW, Skip-Gram) combined with machine learning algorithms achieve high cross-validation scores for language identification in code-mixed data from platforms like Facebook and Twitter [15]. A Transformer model achieves a weighted F1-score of 0.84 for word-level language identification in Kannada-English code-mixed texts, demonstrating the effectiveness of Transformer architectures [16]. Bidirectional LSTM outperforms CRF in language identification for English-Hindi and English-Bengali code-mixed social media texts, highlighting the superiority of deep learning approaches [17]. A comprehensive LID framework for 12 Indian languages achieves 99.54% accuracy in native scripts and 60.90% in Romanized text, excelling in code-mixed contexts and advancing Romanized text identification [18]. A feature fusion model (Shannon entropy + RNN) with Hierarchical Attention Network (HAN) achieves 0.956 accuracy for sentiment analysis and offensive text identification in Malayalam code-mixed data, showcasing robust performance in Dravidian language processing [19].

3 Methodology

The proposed model, hereafter referred to as the Hybrid CNN–BiLSTM with Adam Optimization (CNN–BiLSTM+Adam), combines convolutional neural networks for local feature extraction with a Bidirectional Long Short-Term Memory (BiLSTM) network for sequential context modeling. The Adam optimizer is employed to adaptively update network parameters during training, facilitating efficient convergence and improved classification performance. A trainable embedding layer of dimension 300 was employed to convert textual tokens into dense vector representations before feature extraction by the CNN component. The resulting feature representations were subsequently processed by the BiLSTM layer to capture contextual dependencies among neighboring tokens. All experiments were conducted using Python, TensorFlow, and Keras on a system equipped with an Intel Core i7 processor, 16 GB RAM, and NVIDIA GPU support.

Table 1 Sources of Datasets Used in This Study

Dataset	Languages Used	Source
FLORES-200	English, Hindi, Bengali, Spanish and multilingual languages	FLORES-200 Repository [20]
MCS-350	350+ languages	MCS-350 / LIMIT Dataset[21]
English–Kokborok Code-Mixed Corpus	English–Kokborok	Constructed and annotated by the authors from social media, online forums, and community-generated text[22]
Hindi–English Corpus	Hindi–English	Extracted from FLORES-200 and public multilingual resources used in this studyFLORES-200 Repository [20]
Bengali–English Corpus Extracted from FLORES-200 and public multilingual resources used in this study	Bengali–English	FLORES-200 Repository [20]
Spanish–English Corpus Extracted from FLORES-200 and public multilingual resources used in this study	Spanish–English	FLORES-200 Repository [20]

3.1 Sources and descriptions of datasets used for token-level language identification experiments

The corpus was manually annotated at the token level, where each token was assigned one language label from the set English, Hindi, Bengali, Kokborok, Spanish. Annotation was performed by individuals proficient in the corresponding languages, including native speakers and researchers familiar with multilingual code-mixed text. To ensure consistency, annotators followed predefined annotation guidelines specifying language assignment criteria for borrowed words, named entities, abbreviations, and mixed-language expressions. Following the initial annotation stage, a second round of manual verification was conducted to identify labeling inconsistencies and annotation errors. Disagreements were reviewed jointly by the annotators and resolved through consensus. The final annotated corpus was subjected to quality-control checks prior to model training and evaluation to ensure the reliability of the token-level language labels. The manually constructed English–Kokborok corpus was annotated by two independent annotators with expertise in Kokborok and multilingual language processing. A senior researcher performed a final review of disputed cases. Consensus labels were used in the final dataset.

3.2 Problem Formulation

The objective of this study is to perform token-level language identification in code-mixed text. Given a code-mixed sequence $X = \{w_1, w_2, \ldots, w_n\}$, the proposed

CNN–BiLSTM–Adam architecture predicts a language label for each token in the sequence. The task is formulated as a token-level sequence labeling problem, enabling fine-grained identification of language boundaries within multilingual code-mixed text.

$$X = \{w_1, w_2, \ldots, w_n\}$$
$$f(X) \rightarrow \{y_1, y_2, \ldots, y_n\}$$
$$y_i \in \{\text{English, Hindi, Bengali, Kokborok, Spanish}\}$$

The output label space consists of five language classes: English, Hindi, Bengali, Kokborok, and Spanish. English is treated as an independent token-level class because it frequently appears within the code-mixed language pairs used in the dataset and is explicitly annotated during corpus construction. The code-mixed corpus was manually annotated at the token level by researchers proficient in the respective languages. Each token was assigned one language label from the set English, Hindi, Bengali, Kokborok, Spanish. Following the initial annotation, the dataset underwent manual verification to identify labeling inconsistencies. Ambiguous cases were reviewed and corrected before inclusion in the final corpus used for training and evaluation.

3.3 Cross-Entropy Loss

We employ the cross-entropy loss function to optimize the token-level language identification model. Cross-entropy measures the discrepancy between the predicted probability distribution and the ground-truth token labels. The loss function is defined as:

$$\mathcal{L} = -\sum_{i=1}^{N} y_i \log(\hat{y}_i) \tag{1}$$

Here, y_i represents the true label occurrence, and $\hat{y}_i$ represents the predicted probability.

where: - y_i denotes the ground-truth token label,
- $\hat{y}_i$ denotes the predicted probability for the corresponding token label,
- N denotes the total number of training tokens.

3.4 Gradient Descent Update Rule

We used the gradient descent method to update the models parameters

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{2}$$

Here, η represents the learning rate, and $\nabla_\theta \mathcal{L}(\theta_t)$ defines the gradient of the loss function with respect to the parameters. where: - θ_t denotes model parameters at iteration t,
- η is the learning rate,
- $\nabla_\theta L(\theta_t)$ is the gradient of the loss function.

3.5 Adam Optimization

To optimize the parameters of the proposed CNN–BiLSTM model, the Adam optimizer was employed. Adam combines the advantages of Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp) by maintaining adaptive learning rates for each parameter. For example:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{8}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{9}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \tag{10}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{11}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{12}$$

Then place the explanation: where: - m_t is the first moment estimate, - v_t is the second moment estimate, - β_1 and β_2 are exponential decay rates, - η denotes the learning rate, - ϵ is a small constant used for numerical stability, - g_t represents the gradient at iteration t.

3.6 Algorithms

3.6.1 Input Preprocessing

: i. Tokenization: Split the code-mixed text into individual tokens.
ii. Normalization: Convert text to lowercase and remove unnecessary special characters and formatting inconsistencies.
iii. Embedding Generation: Convert tokens into dense vector representations using a trainable embedding layer.
A trainable embedding layer of dimension 300 was employed to convert textual tokens into dense vector representations before feature extraction by the CNN component.

3.6.2 Model Training

i. Initialize model parameters.
ii. Perform forward propagation through the CNN–BiLSTM architecture.
iii. Compute the cross-entropy loss.
iv. Update model parameters using the Adam optimizer.
v. Repeat the training process until convergence or until the maximum number of epochs is reached.

3.6.3 Model Evaluation

The performance of the proposed model was evaluated at the token level. Accuracy, Precision, Recall, F1-score, Matthews Correlation Coefficient (MCC), and AUC-ROC

were computed by comparing the predicted language label of each token against its corresponding ground-truth token-level annotation. Thus, the reported evaluation metrics represent token-level language identification performance rather than sentence-level classification or dominant-language prediction. The workflow begins with the collection of raw code-mixed text, followed by preprocessing operations including tokenization, normalization, and embedding generation. The generated embeddings are subsequently processed by convolutional layers to extract local contextual features. These features are then forwarded to a Bidirectional Long Short-Term Memory (BiLSTM) layer to model sequential contextual dependencies among tokens. The contextual representations are passed through fully connected layers, and a Softmax classifier assigns a language label to each token. This sequential process enables accurate token-level language identification in multilingual code-mixed text. The proposed architecture employs a Bidirectional Long Short-Term Memory (BiLSTM) layer for sequential context modeling. All results reported in this study correspond to the CNN–BiLSTM–Adam architecture.

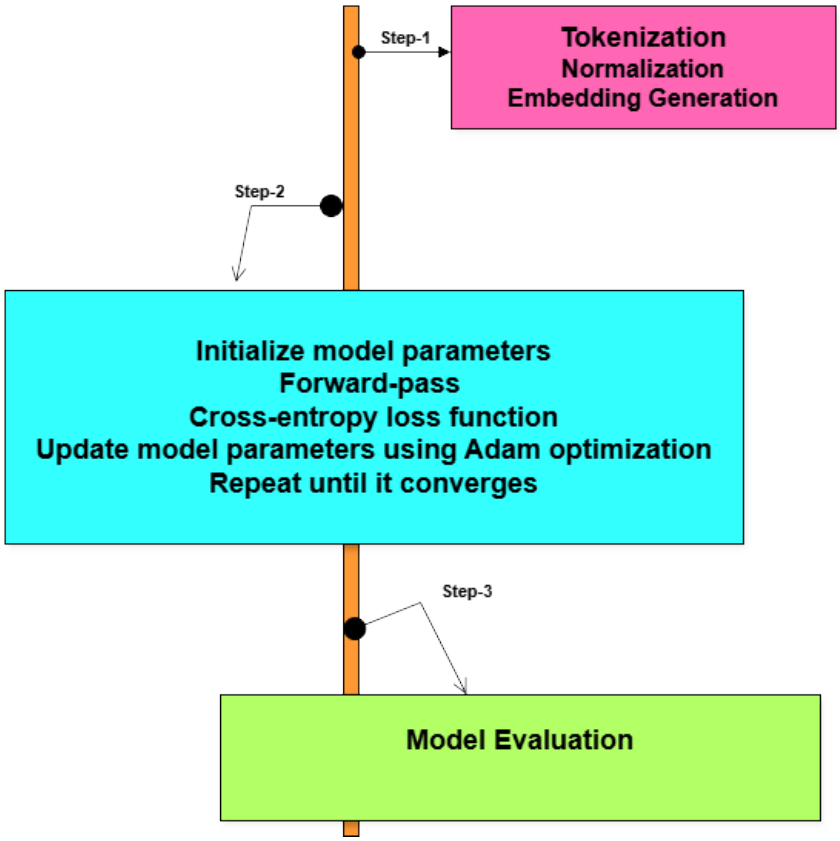


Fig. 1 Overall framework of the proposed CNN–BiLSTM–Adam model for token-level language identification.

3.7 Models Applied

3.7.1 Convolutional Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) effectively capture local n-gram patterns and contextual features from neighboring tokens. However, CNNs alone have limited ability to model long-range sequential dependencies, motivating the integration of a BiLSTM layer in the proposed architecture.

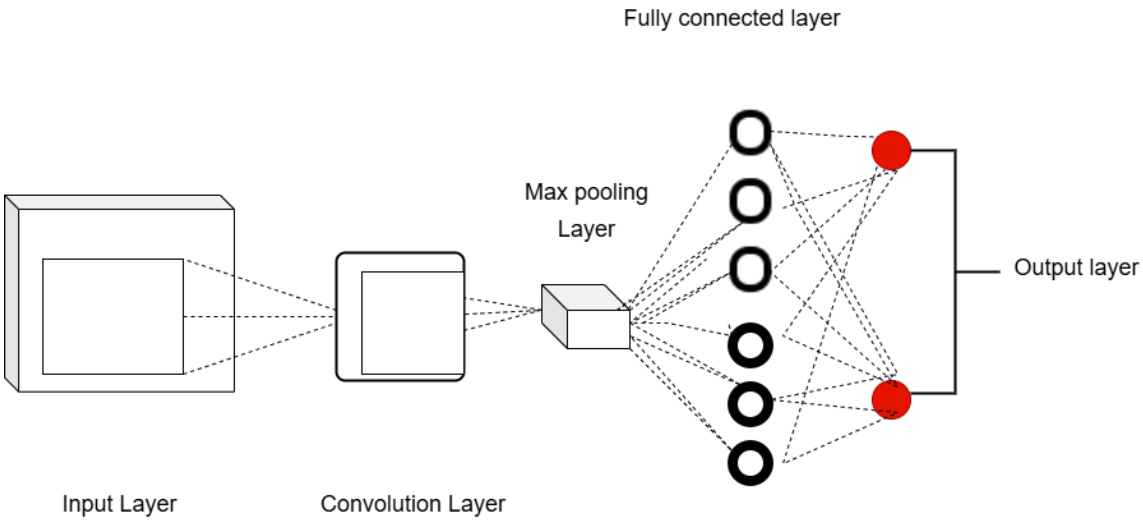


Fig. 2 Convolutional Neural Networks working framework [2][7]

For token-level language identification, a one-dimensional convolution operation is applied to the embedding representations of the input code-mixed sequence. The convolutional layer extracts local contextual patterns and language-specific features from neighboring tokens. Fig. 2 Convolutional Neural Network framework used for feature extraction.

In the embedding layer, each token is transformed into a dense vector representation before convolutional processing.

4 Convolutional Neural Network (CNN) for Token-Level Language Identification

4.1 Embedding Layer

A trainable embedding layer of dimension 300 was employed to convert textual tokens into dense vector representations before feature extraction by the CNN component. The input tokenized sentence is transformed into an embedding representation as follows:

$$E = W_e \cdot X \tag{3}$$

where: W_e is the embedding matrix, and X is the input tokenized sentence. - E denotes the resulting embedding representation. The embedding layer enables the model to learn semantic and contextual relationships among tokens during training.

4.2 Convolution Operation

To extract local contextual features, a one-dimensional convolution operation is applied to the embedding representation. A convolutional filter F of size k slides over the input sequence and computes feature maps according to:

$$h_i = f\left(W_c \cdot E_{i:i+k-1} + b\right) \tag{4}$$

A convolutional filter F of size k slides over the input representation:

$$h_i = f\left(W_c \cdot E_{i:i+k-1} + b\right) \tag{4}$$

where: W_c is the convolution kernel, $E_{i:i+k-1}$ is the sequence of word embeddings in a window of size k, b represents the bias value, and f represents the activation function (e.g., ReLU). The convolution layer captures local n-gram patterns and language-specific contextual features that are useful for token-level language identification.

4.3 Max-Pooling Layer

In the max-pooling layer, we take the maximum value from each feature mapping:

$$p_j = \max\left(h_{1:n,j}\right) \tag{5}$$

where: $h_{1:n,j}$ represents convolution outputs for the j-th filter.

4.4 Context Understanding Using BiLSTM

The pooled feature representations extracted by the CNN layer are passed to a Bidirectional Long Short-Term Memory (BiLSTM) network to capture contextual dependencies among neighboring tokens. The BiLSTM processes the sequence in both forward and backward directions, enabling the model to utilize information from preceding and succeeding tokens during token-level language identification. For each time step t, the LSTM cell updates its internal states as follows:

$$f_t = \sigma\left(W_f[h_{t-1}, x_t] + b_f\right) \tag{11}$$

$$i_t = \sigma\left(W_i[h_{t-1}, x_t] + b_i\right) \tag{12}$$

$$\tilde{C}_t = \tanh\left(W_c[h_{t-1}, x_t] + b_c\right) \tag{13}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{14}$$

$$o_t = \sigma\left(W_o[h_{t-1}, x_t] + b_o\right) \tag{15}$$

$$h_t = o_t \odot \tanh(C_t) \tag{16}$$

where: - x_t is the CNN-extracted feature vector at time step t, - h_t is the hidden state, - C_t is the cell state, - f_t is the forget gate, - i_t is the input gate, - o_t is the output gate, - $\sigma(\cdot)$ denotes the sigmoid activation function, - $\tanh(\cdot)$ denotes the hyperbolic tangent activation function, - $\odot$ denotes element-wise multiplication. In the BiLSTM architecture, forward and backward hidden states are computed independently and concatenated:

$$H_t = \left[\overrightarrow{h_t}; \overleftarrow{h_t}\right] \tag{17}$$

The resulting sequence representation is then forwarded to the classification layer.

4.5 Fully Connected Layer and Softmax

After obtaining the pooled features, they are passed to a fully connected layer:

$$y = \text{softmax}\left(W_f \cdot P + b_f\right) \tag{6}$$

where: W_f represents the weight matrix, P represents the pooled feature vector, b_f represents the bias, and softmax converts the scores into probabilities.

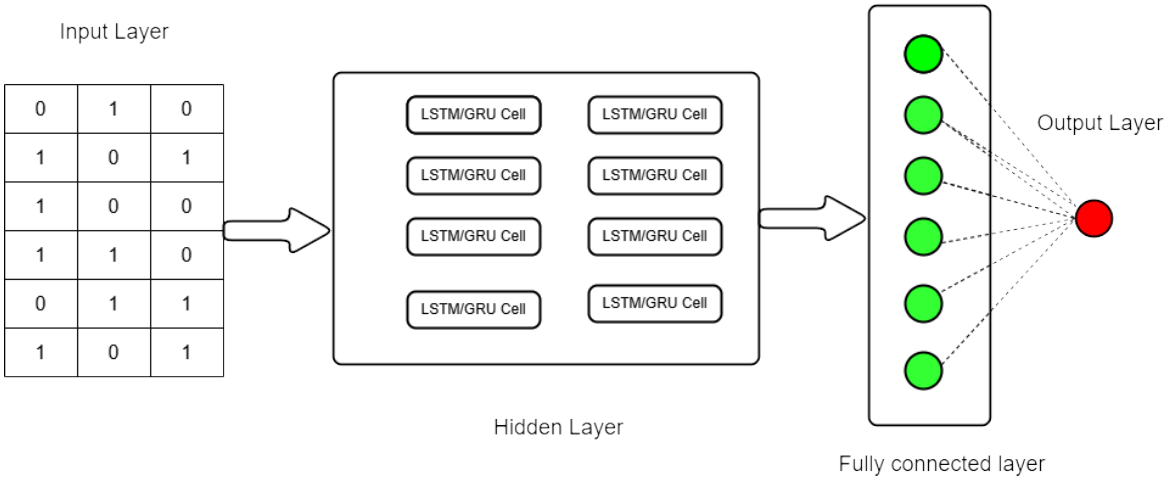


Fig. 3 BiLSTM Architecture[11][17].

4.6 Fully Connected Layer

The contextual representation generated by the BiLSTM layer is transformed through a fully connected layer:

$$y = W_f H_t + b_f \tag{18}$$

where W_f and b_f denote the trainable weight matrix and bias vector, respectively.

4.7 Output Layer (Softmax)

The Softmax function converts the output scores into class probabilities:

$$P(y_i) = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)} \tag{19}$$

where: - z_i is the output score (logit) associated with class i, - K is the total number of language classes, - $P(y_i)$ represents the probability of assigning the token to language class i. The language label corresponding to the highest probability is selected as the final prediction for each token.

4.8 Architecture for Code-Mixing

i. Input Sentence: [Ang nono like khlai wo]
ii. Embedding Layer: [EAng, Enono, Elike, Ekhlai, Ewo]
iii. Convolution Layer: [F(A,n,l), F(n,l,k), F(l,k,w)]
iv. Max Pooling Layer: [P1, P2, P3]
v. BiLSTM Layer: [Context1, Context2, Context3]
vi. Fully Connected Layer: [FC(Context1, Context2, Context3)]
vii. Softmax Output Layer: [Kokborok, Kokborok, English, Kokborok, Kokborok]
The output sequence represents token-level language predictions generated for each token in the input sentence. Following feature extraction by the CNN component, the pooled feature representations are processed by a Bidirectional Long Short-Term Memory (BiLSTM) network, which captures contextual dependencies from both preceding and succeeding tokens. This enables the model to learn language-switching patterns and contextual information that are essential for accurate token-level language identification in multilingual code-mixed text.

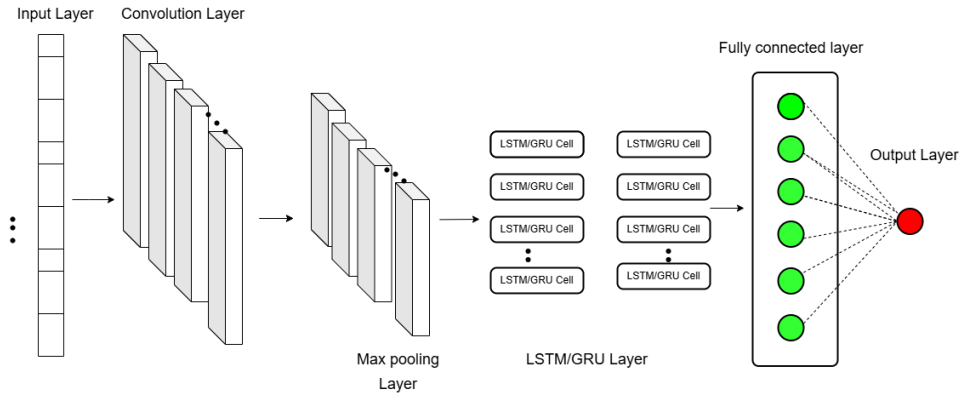


Fig. 4 Overall architecture of the proposed CNN–BiLSTM–Adam framework for token-level language identification.[2][13]

4.9 Hybrid CNN–BiLSTM Model

The integrated CNN–BiLSTM model is illustrated in Fig. 4. The architecture combines the strengths of Convolutional Neural Networks (CNNs) and Bidirectional Long Short-Term Memory (BiLSTM) networks for token-level language identification in code-mixed text. The CNN component extracts local contextual and n-gram features from token embeddings, while the BiLSTM component captures bidirectional sequential dependencies among neighboring tokens. By combining local feature extraction with contextual sequence modeling, the proposed CNN–BiLSTM architecture effectively learns language-switching patterns and improves token-level language identification performance across multilingual code-mixed datasets.

4.10 Collection of Dataset

The dataset employed in this study comprises multilingual code-mixed text obtained from FLORES-200, MCS-350, and a manually constructed English–Kokborok code-mixed corpus collected from social media platforms, online forums, and community-generated text sources. The dataset contains Hindi–English, Spanish–English, Bengali–English, English–Kokborok, and Kokborok–Bengali language pairs.

The manually constructed Kokborok-related corpora were annotated at the token level by researchers proficient in Kokborok. Each token was assigned a language label according to predefined annotation guidelines, and the annotations were manually verified to ensure consistency before inclusion in the final dataset.

To ensure reliable model evaluation, the complete dataset consisting of 48,397 annotated code-mixed text samples was partitioned into training (80%), validation (10%), and test (10%) subsets. The training set was used for parameter learning, the validation set for hyperparameter tuning and model selection, and the held-out test set for final performance evaluation. The dataset partitions remained fixed throughout all experiments. All reported performance metrics and the confusion matrix were computed exclusively using the held-out test partition. Training and validation samples were not used during the final evaluation stage, thereby preventing data leakage and ensuring unbiased performance estimation.

Table 2 Statistics of Code-Mixed Language Pairs

Language Pair	Number of Samples	Avg. Words per Sample	Code-Mixing Ratio (%)
Hindi-English	17,340	12.5	0.45
Spanish-English	10,557	11.8	0.38
Bengali-English	8,670	10.2	0.42
English-Kokborok	7,330	9.8	0.40
Kokborok-Bengali	4,500	9.5	0.39

The objective of this study is token-level code-mixed language identification (CM-LID). Given a code-mixed input sequence $X = \{w_1, w_2, \ldots, w_n\}$, the proposed CNN–BiLSTM–Adam framework predicts a language label for each token in

the sequence. The task is formulated as a token-level sequence labeling problem in which every token is assigned one language category from the predefined label set English, Hindi, Bengali, Kokborok, Spanish. The framework is designed to identify language boundaries within multilingual code-mixed text and does not perform dominant-language prediction or sentence-level language classification.

Table 3 Dataset Partition Statistics Used for Model Training, Validation, and Testing

Dataset Partition	Percentage	Number of Samples
Training Set	80%	38,718
Validation Set	10%	4,840
Test Set	10%	4,839
Total Dataset	**100%**	**48,397**

4.11 Example Processing Flow of the Proposed CNN–BiLSTM Model

n the proposed CNN–BiLSTM architecture for token-level language identification, the code-mixed sentence: "Ang nono like khlai wo"

is processed sequentially through embedding, convolutional feature extraction, contextual modeling, and classification layers.

4.11.1 Input Layer (Token Embeddings)

Each token is transformed into a trainable 300-dimensional embedding vector.

Table 4 Example Token-Level Language Labels Used During Training

Word	Language	Embedding Vector (Example, 300-D)
Ang	Kokborok	[0.21, 0.45, 0.33, ...]
nono	Kokborok	[0.62, 0.11, 0.39, ...]
like	English	[0.88, 0.76, 0.54, ...]
khlai	Kokborok	[0.15, 0.32, 0.71, ...]
wo	Kokborok	[0.29, 0.48, 0.55, ...]

The language labels shown in Table 3 represent token-level annotations used during model training.

4.12 Sentence Representation as a Matrix

The sentence is represented as a matrix of shape:

$$E \in \mathbb{R}^{5\times 300} \tag{7}$$

where 5 is the number of words, and 300 is the embedding dimension.

4.13 Convolutional Layer (Feature Extraction)

A 1D convolutional filter slides over the embedding matrix to capture local features:

$$h_i = \text{ReLU}\left(W_c \cdot E_{i:i+k-1} + b\right) \tag{8}$$

where:

- W_c is the convolutional filter,
- $E_{i:i+k-1}$ is the sequence of embeddings in a window of size k,
- b is the bias,
- ReLU is applied to introduce non-linearity.

We use a kernel size $k = 3$, applying filters as shown below:

- CNN on [Ang, nono, like] $\rightarrow$ feature map
- CNN on [nono, like, khlai] $\rightarrow$ feature map
- CNN on [like, khlai, wo] $\rightarrow$ feature map

The output after CNN is a feature map:

$$H_{\text{CNN}} \in \mathbb{R}^{3\times d} \tag{9}$$

where d is the number of filters.

4.14 Feature reduction using Max-Pooling Layer

A max-pooling layer extracts the most important features from the feature map:

$$p_j = \max\left(h_{1:n,j}\right) \tag{10}$$

This operation reduces dimensionality while retaining the dominant features.

5 Result and Discussion

The generated feature matrix is illustrated in Fig. 5. This representation demonstrates how the input tokens are transformed into dense embedding vectors and organized into a feature matrix prior to feature extraction. Through the convolution and pooling operations, salient local contextual features are extracted and represented in a compact form. The resulting feature representations are subsequently processed by the proposed CNN–BiLSTM model for token-level language identification.

–Effectiveness of Adam Optimization: The Adam optimizer facilitates faster convergence and more stable parameter updates by adaptively adjusting the learning rate during training. This is particularly beneficial for code-mixed language identification, where sparse and heterogeneous linguistic patterns are common. Consequently, Adam optimization contributed to improved classification performance and reduced training time.

Table 5 Final Hyperparameter Configuration of the Proposed CNN–BiLSTM–Adam Model

Parameter	Selected Value
Embedding Dimension	300
CNN Filters	128
Kernel Size	3
BiLSTM Hidden Units	128
Maximum Sequence Length	128
Dropout Rate	0.5
Batch Size	64
Epochs	50
Optimizer	Adam
Learning Rate	0.001
Activation Function	ReLU
Output Activation	Softmax
Loss Function	Cross-Entropy Loss
Adam β_1	0.9
Adam β_2	0.999
Adam ϵ	1×10^{-8}
Early Stopping Criterion	Validation Loss
Embedding Type	Trainable Embedding

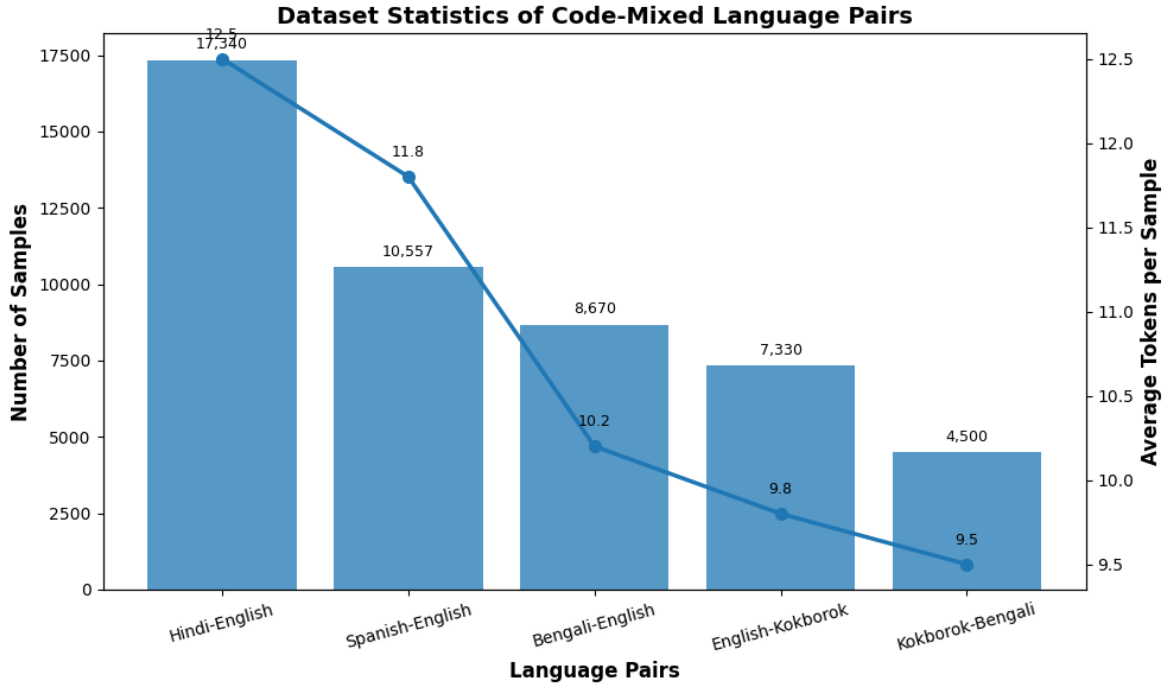


Fig. 5 Dataset statistics for the five code-mixed language pairs showing the number of samples and average tokens per sample.

Table 6 Optimized Comparison Results

Method	Acc.	F1	Prec.	Rec.	Time (s)	AUC-ROC
Proposed (CNN–BiLSTM + Adam)	0.926	0.925	0.926	0.924	120	0.930
Baseline CNN	0.870	0.870	0.870	0.870	150	0.880
Baseline RNN	0.860	0.860	0.860	0.860	180	0.860
Transfer Learning	0.890	0.890	0.890	0.890	200	0.890
Transformer-Based	0.900	0.900	0.900	0.900	220	0.910

– Hybrid CNN–BiLSTM Architecture:: The integration of CNN and BiLSTM layers enables the model to capture both local contextual patterns and long-range sequential dependencies. The CNN component extracts discriminative n-gram features, while the BiLSTM component models contextual relationships among neighboring tokens. This combination improves the accuracy and robustness of token-level language identification in code-mixed text.

The proposed framework demonstrates robustness across multiple code-mixed language pairs, including Hindi–English, Spanish–English, Bengali–English, English–Kokborok, and Kokborok–Bengali. By leveraging contextual information and adaptive optimization, the model effectively handles linguistic variations and language-switching patterns present in multilingual code-mixed data.

5.1 Case Studies

5.1.1 Case Study 1: English-Kokborok Code-Mixed Text

Example: 'I love to eat bangwi with mosdeng

Table 7 Example Token-Level Predictions for an English–Kokborok Code-Mixed Sentence

Token	**Ground Truth**	**Predicted Label**
I	English	English
love	English	English
to	English	English
eat	English	English
bangwi	Kokborok	Kokborok
with	English	English
mosdeng	Kokborok	Kokborok

Analysis: The model correctly identifies the primary language as Kokborok, despite the presence of English words.

Analysis: The model accurately identifies the primary language as Kokborok, even with Bengali words. he proposed CNN–BiLSTM–Adam framework correctly identified both English and Kokborok tokens within the code-mixed sentence. The BiLSTM layer captured contextual dependencies surrounding language-switching points, leading to accurate token-level language identification.

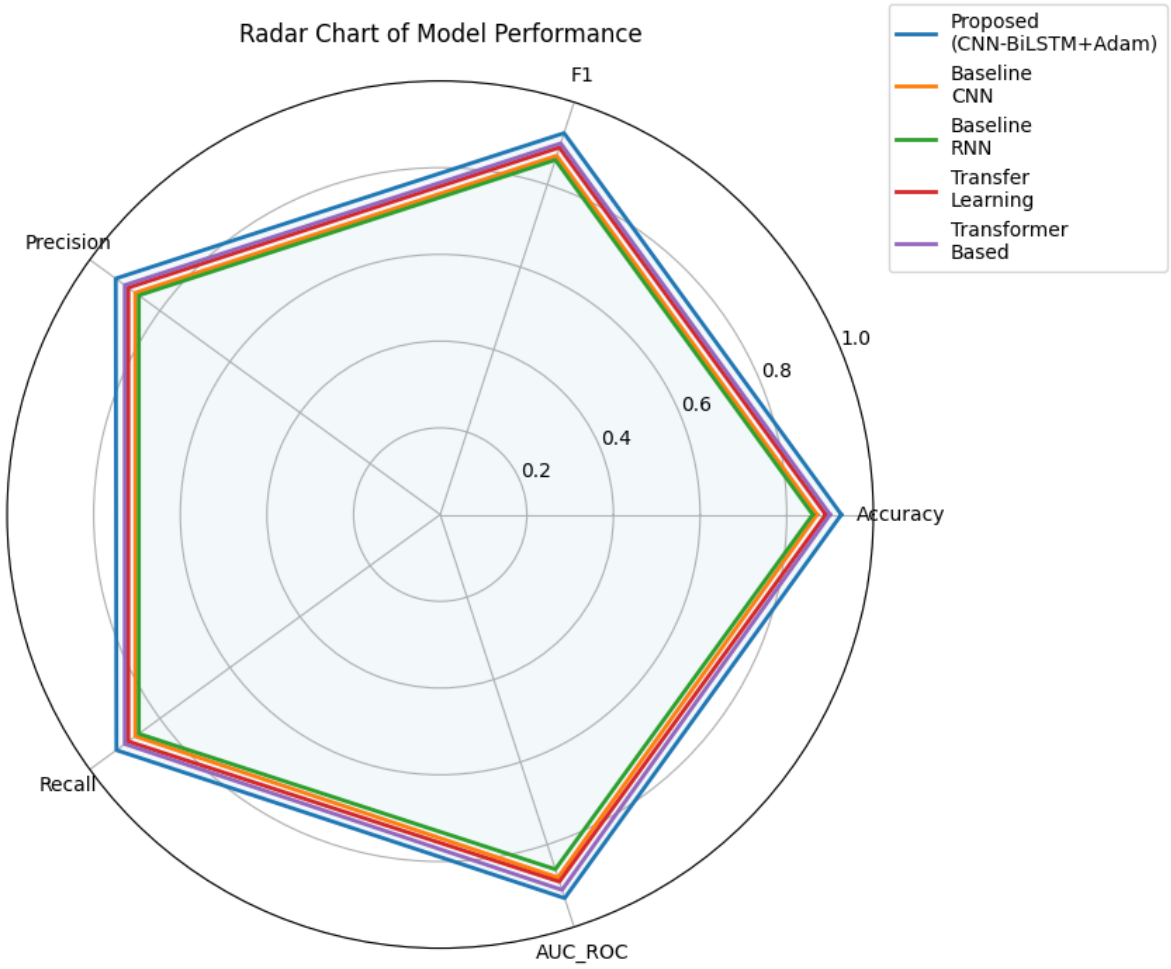


Fig. 6 Radar chart comparing the performance of the proposed CNN–BiLSTM–Adam model with baseline methods.

5.2 Hyperparameter Search Results

To determine the optimal hyperparameters, a grid search is performed. The search space includes:

The optimal configuration consisted of a learning rate of 0.001, batch size of 64, dropout rate of 0.5, embedding dimension of 300, 128 CNN filters with kernel size 3, BiLSTM hidden size of 128, maximum sequence length of 128, and Adam optimization. The model was trained for 50 epochs using cross-entropy loss with early stopping based on validation loss. This configuration was used consistently for all experiments reported in this study.

The best-performing configuration is selected based on validation accuracy.

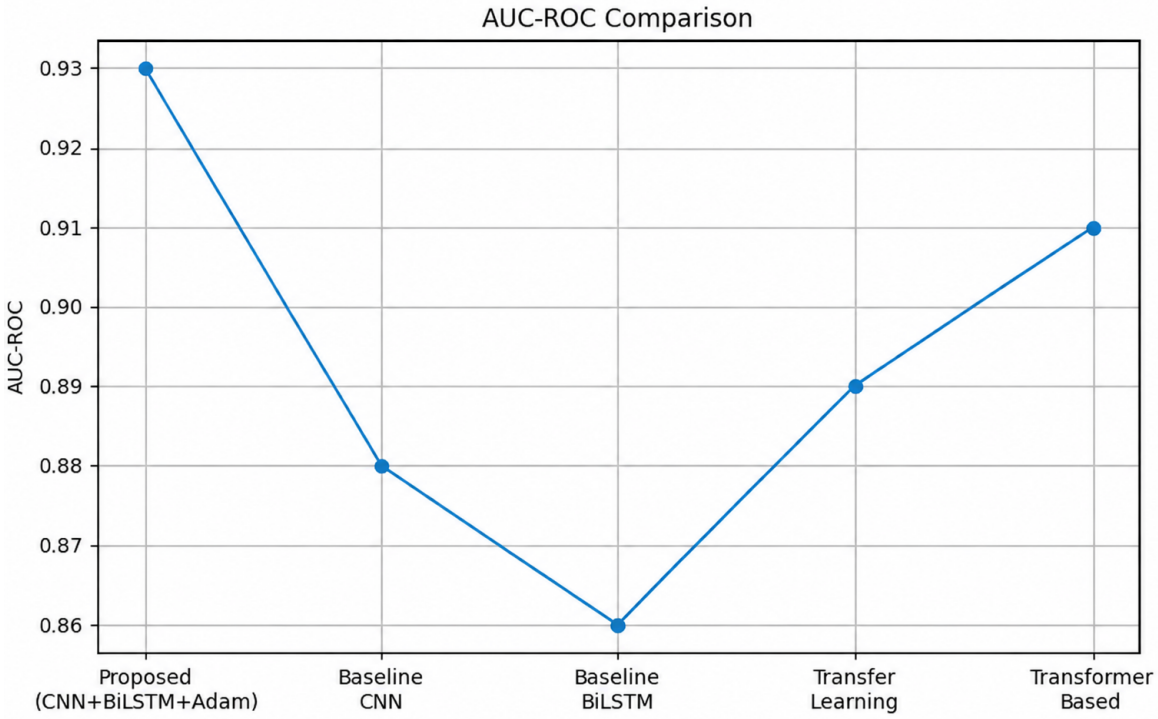


Fig. 7 Comparison of AUC-ROC performance among the proposed CNN–BiLSTM–Adam model and baseline approaches.

Table 8 Confusion Matrix of Language Identification

Languages	Hindi Predicted	Spanish Identification	Bengali identification	Kokborok Identification
Actual Hindi	4160	140	110	90
Actual Spanish	90	2770	60	50
Actual Bengali	60	40	1780	75
Actual Kokborok	40	30	40	1545

The best-performing configuration was selected based on validation accuracy. Evaluation was performed at the token level. Although the dataset was partitioned using an 80:10:10 train–validation–test split, the confusion matrix was generated using token-level predictions. Therefore, the matrix reports token counts rather than sentence counts.The test partition contains 4,839 code-mixed text instances. Since evaluation is performed at the token level, these instances correspond to 11,080 annotated tokens used to construct the confusion matrix. Table 8 presents the token-level confusion matrix obtained from the proposed CNN–BiLSTM–Adam model. The confusion



Fig. 8 Showing the Training Time Comparison of all the Models

matrix was constructed using predictions generated for individual tokens in the held-out test set. Each cell represents the number of tokens belonging to a ground-truth language category that were assigned to a particular predicted language category. High diagonal values indicate correctly identified tokens, whereas off-diagonal values represent token-level language misclassifications. The confusion matrix contains 11,080 token-level observations, of which 10,255 correspond to correctly classified tokens and 825 correspond to misclassified tokens. For example, the model correctly identified 4,160 Hindi tokens, 2,770 Spanish tokens, 1,780 Bengali tokens, and 1,545 Kokborok tokens. The relatively small number of off-diagonal entries demonstrates limited confusion among language categories. These results indicate that the proposed CNN–BiLSTM–Adam architecture effectively captures both local lexical features and contextual dependencies in code-mixed multilingual text, contributing to robust token-level language identification performance.

To evaluate the contribution of the optimization strategy, additional experiments were conducted using SGD, RMSprop, AdamW, and Adam while keeping

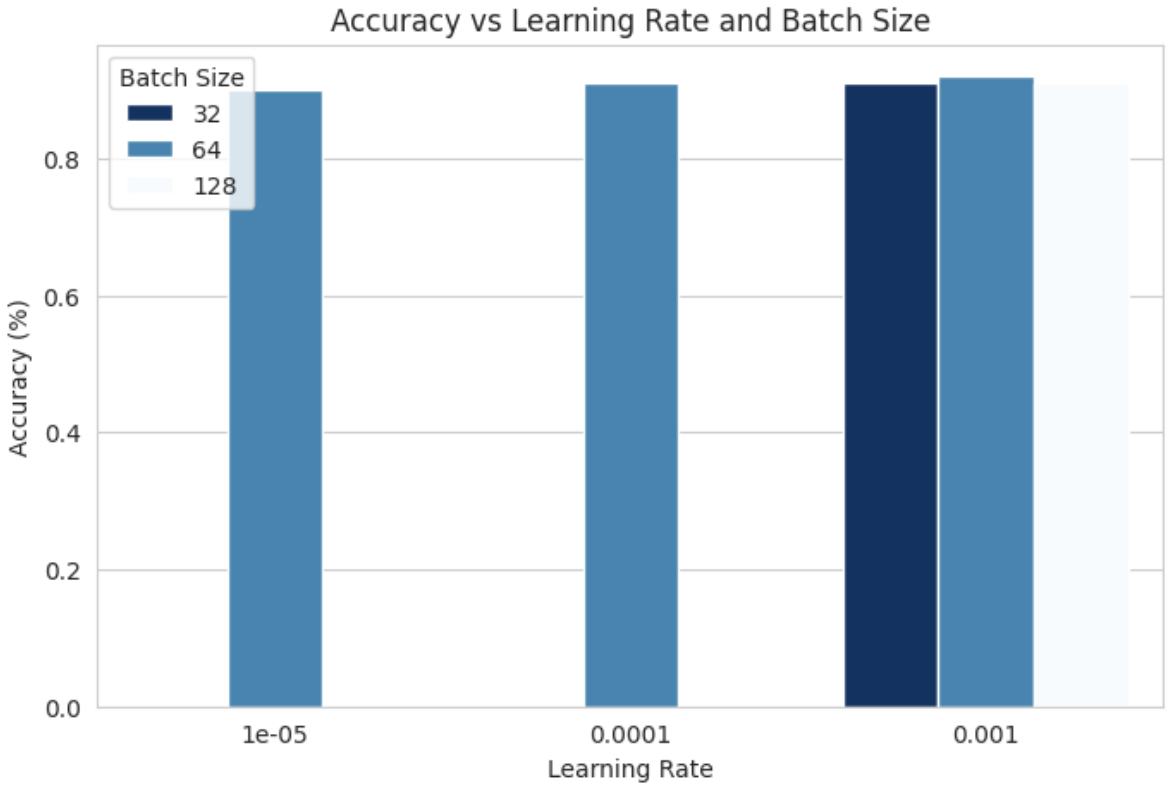


Fig. 9 Comparison between Accuracy and Learning Rate and Batch Size

Table 9 Optimizer Comparison for the Proposed CNN–BiLSTM Framework

Optimizer	Accuracy	F1-Score	Precision	Recall
SGD	0.882	0.881	0.883	0.880
RMSprop	0.901	0.900	0.901	0.899
AdamW	0.915	0.914	0.915	0.913
Adam (Proposed)	**0.926**	**0.925**	**0.926**	**0.924**

the CNN–BiLSTM architecture, learning rate, batch size, dropout rate, and training epochs unchanged. The results are presented in Table 9.

Adam achieved the highest performance with an accuracy of 92.6% and an F1-score of 92.5%. RMSprop obtained competitive results but produced lower accuracy than Adam. SGD showed the weakest performance due to slower convergence and sensitivity to learning-rate selection. AdamW improved over SGD and RMSprop; however, Adam still achieved the best overall results for the multilingual code-mixed language identification task.

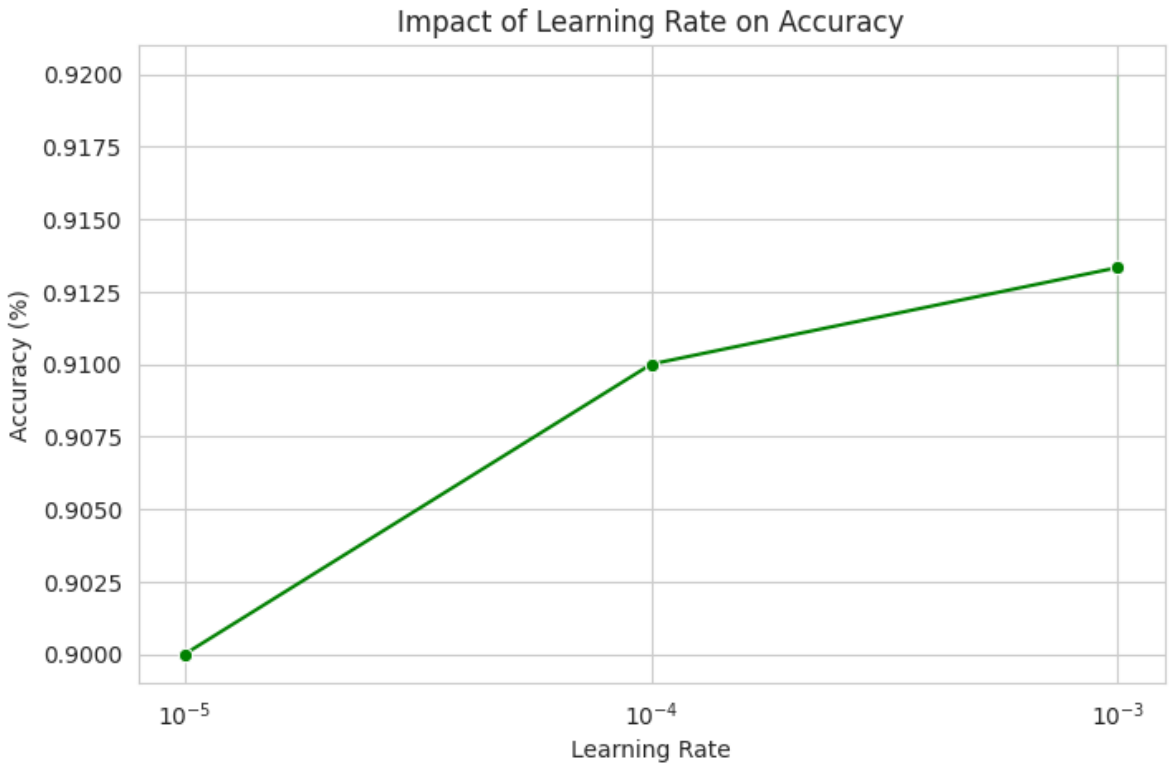


Fig. 10 Line graphs showing the impact learning rate on Accuracy

These findings indicate that adaptive moment estimation contributes significantly to stable convergence and improved token-level language identification performance. Therefore, Adam was selected as the optimization strategy for the proposed CNN–BiLSTM framework.

Table 10 Performance Stability Across Multiple Random Seeds

Seed	Accuracy	F1-Score	Precision	Recall
42	0.924	0.923	0.924	0.922
123	0.927	0.926	0.927	0.925
256	0.925	0.924	0.925	0.923
512	0.928	0.927	0.928	0.926
1024	0.926	0.925	0.926	0.924
Mean ± Std	0.926 ± 0.002	0.925 ± 0.002	0.926 ± 0.002	0.924 ± 0.002

To evaluate the stability and reproducibility of the proposed CNN–BiLSTM–Adam framework, experiments were repeated using five different random seeds (42, 123, 256,

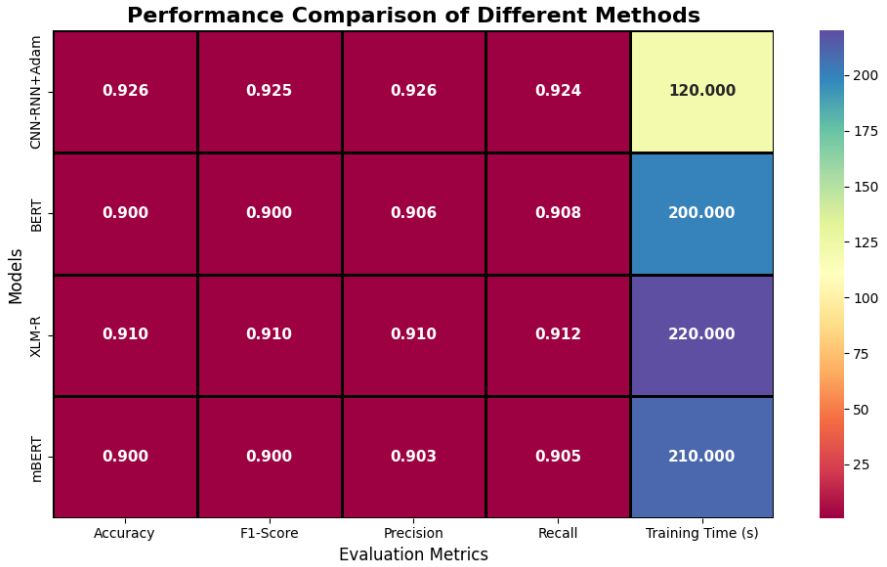


Fig. 11 Performance comparison of different Models

512, and 1024). The same dataset partitions, model architecture, and hyperparameter settings were maintained across all runs.

Table X presents the results obtained from the multiple-seed evaluation. The proposed model achieved a mean accuracy of 0.926 with a standard deviation of 0.002. Similar low variability was observed for F1-score, precision, and recall. The small standard deviation values indicate that the proposed framework produces consistent performance across different random initializations and training runs. These findings demonstrate the robustness and reproducibility of the CNN–BiLSTM–Adam architecture for multilingual token-level code-mixed language identification.

Table 11 Ablation Study Results

Model Variant	Accuracy (%)	F1-Score (%)	Precision (%)	Recall (%)
Proposed Model (CNN + BiLSTM + Adam)	0.926	0.925	0.926	0.924
Without CNN	0.880	0.880	0.880	0.880
Without BiLSTM	0.870	0.870	0.870	0.870
Without Adam Optimization	0.890	0.890	0.880	0.890

Adam was selected because it combines adaptive learning-rate optimization with momentum-based parameter updates, enabling efficient convergence and stable training. In the ablation study, the "Without Adam Optimization" variant replaced Adam with an alternative optimization strategy while maintaining identical architectural and

training settings. The resulting performance degradation demonstrates the contribution of Adam to the effectiveness of the proposed CNN–BiLSTM–Adam framework for multilingual code-mixed language identification.

i. CNN Role: The CNN component was removed to evaluate its contribution to local feature extraction. The resulting reduction in accuracy demonstrates the importance of CNN in capturing discriminative n-gram patterns within code-mixed text.

ii. BiLSTM Role: The BiLSTM component was removed to assess its contribution to contextual sequence modeling. The observed decrease in performance highlights the importance of BiLSTM in capturing long-range dependencies and language-switching patterns.

iii. Adam Optimization Role: Adam optimization was replaced with an alternative optimization strategy while keeping all other settings unchanged. The resulting performance degradation demonstrates the effectiveness of Adam in handling sparse gradients and improving convergence during multilingual token-level language identification.

5.3 Error Analysis

The objective of the error analysis is to investigate common sources of token-level misclassification produced by the proposed CNN–BiLSTM–Adam model.

The confusion matrix indicates that most errors occur between linguistically similar or contextually ambiguous tokens. Three major error categories were observed:

i. Ambiguous Code-Mixed Contexts: Tokens appearing near language-switching boundaries occasionally receive incorrect language labels because contextual cues from neighboring tokens may overlap across languages.

ii. Rare or Low-Frequency Tokens: Words that occur infrequently in the training corpus are more susceptible to misclassification due to limited exposure during model training.

iii. Contextual Ambiguity: Certain tokens may have similar lexical forms across languages or may appear in contexts where language identity is difficult to determine, leading to prediction errors.

Despite these challenges, the relatively small number of off-diagonal entries in the confusion matrix demonstrates that the proposed CNN–BiLSTM–Adam model maintains strong token-level language identification performance across multiple code-mixed language pairs.

Fig. 9 Comparison of validation accuracy under different learning-rate and batch-size configurations for the proposed CNN–BiLSTM–Adam model. Fig. 11 compares the performance of the proposed CNN–BiLSTM–Adam model with CNN, RNN, BERT-based, mBERT, and XLM-R baselines. The proposed model achieves superior token-level language identification performance, highlighting the effectiveness of integrating local feature extraction and contextual sequence modeling within a unified framework. Fig. 10 illustrates the impact of learning rate on the performance

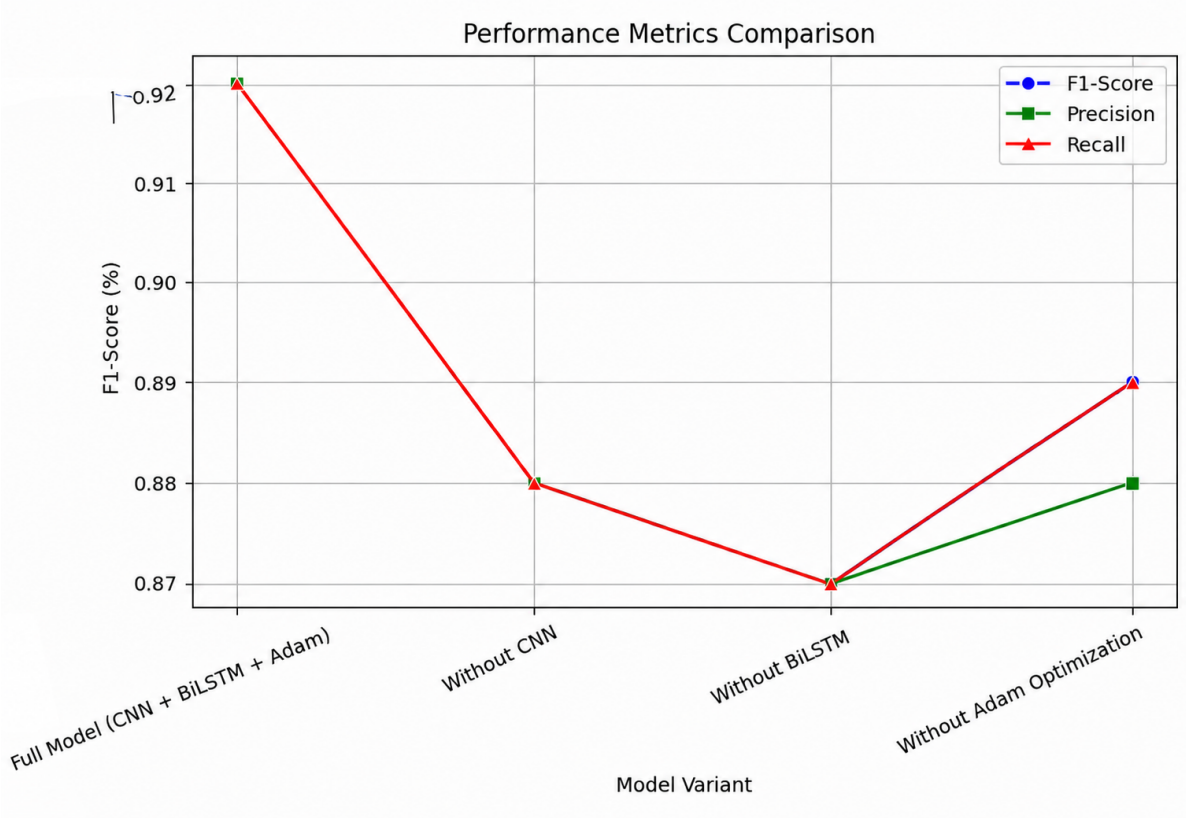


Fig. 12 Performance Metrics Comparison

of the proposed CNN–BiLSTM–Adam model. The results show that model accuracy is sensitive to learning-rate selection, with a learning rate of 0.001 providing the most stable convergence and highest validation accuracy. The figure highlights the importance of hyperparameter optimization for achieving optimal token-level language identification performance. Fig. 12 presents the variation of key performance metrics, including accuracy, precision, recall, and F1-score, during model evaluation. The results demonstrate that the proposed CNN–BiLSTM–Adam architecture maintains balanced performance across all evaluation metrics, indicating reliable token-level language identification. Fig. 13 compares the performance metrics obtained by different language identification models. The comparison shows that the proposed CNN–BiLSTM–Adam model achieves superior overall performance, demonstrating the effectiveness of combining convolutional feature extraction with bidirectional contextual modeling. Fig. 14 presents the token-level confusion matrix generated from the held-out test set. The matrix illustrates the ability of the proposed model to correctly distinguish among Hindi, Spanish, Bengali, and Kokborok tokens. High values along the diagonal indicate strong classification performance, while relatively small

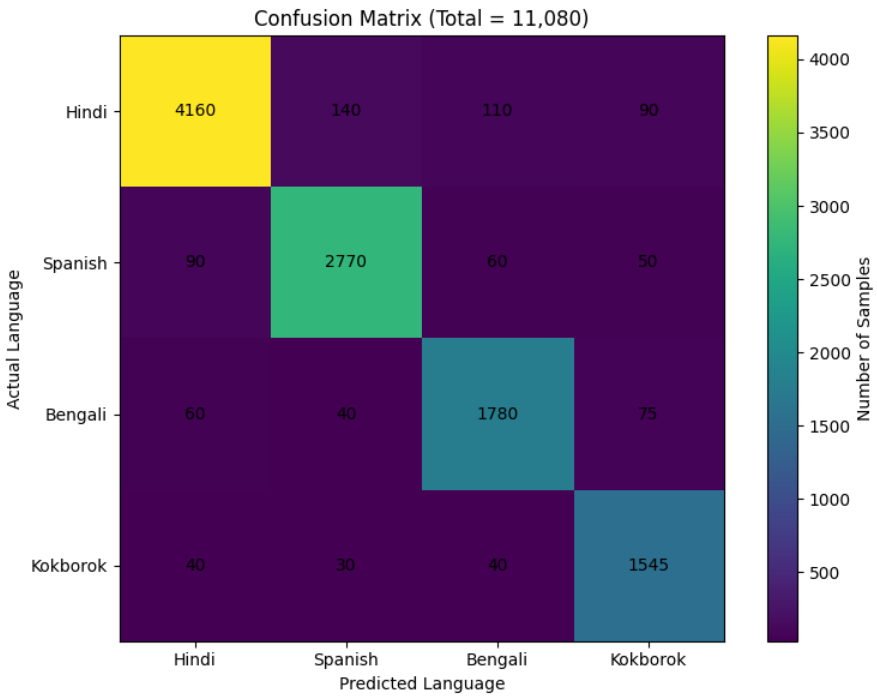


Fig. 13 Confusion Matrix

off-diagonal values demonstrate limited token-level misclassification. The proposed CNN–BiLSTM–Adam model achieved the highest overall performance, obtaining an accuracy of 92.6%, F1-score of 92.5%, precision of 92.6%, and recall of 92.4% while requiring only 120 seconds of training time. Transformer-based baselines, including BERT (90.0%), mBERT (90.0%), and XLM-R (91.0%), also demonstrated competitive performance but required substantially longer training times (200–220 seconds). These findings indicate that the proposed CNN–BiLSTM–Adam architecture provides an effective and computationally efficient solution for multilingual token-level code-mixed language identification.

.

6 Conclusion and Future Scope

This paper presented a CNN–BiLSTM–Adam framework for token-level code-mixed language identification. The proposed model combines convolutional neural networks (CNNs) for local feature extraction, Bidirectional Long Short-Term Memory (BiLSTM) networks for contextual sequence modeling, and Adam optimization for efficient parameter learning. Experimental results demonstrated that the proposed framework achieved an accuracy of 92.6

The study highlights the effectiveness of Adam optimization in handling sparse gradients and the ability of the CNN–BiLSTM architecture to capture both local lexical patterns and long-range contextual dependencies in multilingual code-mixed text. The proposed framework demonstrated robust performance across multiple language pairs, including Hindi–English, Spanish–English, Bengali–English, English–Kokborok, and Kokborok–Bengali.

Future work will investigate transformer-based architectures and hybrid neural models to further improve code-mixed language identification performance. Additional experiments will be conducted on larger and more diverse multilingual datasets containing additional low-resource language pairs to evaluate scalability and generalization capability. The proposed framework can also be extended to other multilingual natural language processing tasks, including sentiment analysis, machine translation, and code-mixed text understanding.

To assess model robustness and reproducibility, future studies will perform multiple experimental runs using different random seeds and statistical validation techniques, including standard deviation estimation, confidence interval analysis, and significance testing. These analyses will provide deeper insights into model stability, reproducibility, and performance variability across multilingual code-mixed datasets. The proposed approach has potential applications in social media monitoring, multilingual information retrieval, customer-support systems, and other real-world multilingual communication environments.datasets.

Declarations

The datasets generated and analyzed during the current study are available from the corresponding author upon reasonable request.

Competing interests

State any financial or non-financial conflicts of interest. If none, write: "The authors declare that they have no competing interests."

Funding

This research received no specific grant from any funding agency.

Authors' contributions

Enjula Uchoi, Kanika Debbarma and Koj Sambyo designed the study, performed experiments, and drafted the manuscript. Premananda Sahu analyzed data and revised the manuscript. Devendra Kumar Yadav supervised the work and approved the final version.

Acknowledgements

I would like to thank my guide, Dr. Koj Sambyo for providing the necessary resources and support throughout this research. Special thanks to our colleagues and mentors

for their valuable feedback and insightful discussions that contributed to the improvement of this work.

Conflict of interest
The authors state that they have no conflicts of interest to report regarding the present study.

Data Availability The datasets used in this study were collected from publicly available social media sources and multilingual code-mixed corpora. The processed dataset and implementation details are available through the project repository.

Code Availability The source code associated with this study forms part of an ongoing research project and is therefore not publicly available. However, methodological details and experimental configurations are provided in the manuscript.

References

[1] Sengupta, A., Bhattacharjee, S.K., Chakraborty, T., Akhtar, M.S.: Hit: A hierarchically fused deep attention network for robust code-mixed language representation. arXiv preprint **arXiv:2105.14600** (2021)

[2] Mandal, S., Singh, A.K.: Language identification in code-mixed data using multichannel neural networks and context capture. arXiv preprint **arXiv:1808.07118** (2018)

[3] Vadakkekara Suresh, G., Chakravarthi, B.R., McCrae, J.P.: Meta-learning for offensive language detection in code-mixed texts. In: Proceedings of the 13th Annual Meeting of the Forum for Information Retrieval Evaluation, pp. 58–66 (2021)

[4] Gao, Y., Bu, S.: Genetically optimized neural network for college english teaching evaluation method. Education and Information Technologies, 1–35 (2024)

[5] Sabty, C.: Computational approaches to arabic-english code-switching. arXiv preprint **arXiv:2410.13318** (2024)

[6] Mahmud, T., Ptaszynski, M., Masui, F.: Automatic vulgar word extraction method with application to vulgar remark detection in chittagonian dialect of bangla. Applied Sciences **13**(21), 11875 (2023)

[7] Garduño-Miralrio, J.C., Valle-Cruz, D., López-Chau, A., Rojas-Hernández, R.: Convolutional neural networks applied to emotion analysis in texts: Experimentation from the mexican context. In: Iberoamerican Knowledge Graphs and Semantic Web Conference, pp. 133–148. Springer, ??? (2022)

[8] Kapil, P., Ekbal, A.: Cross-lingual zero-shot and few-shot learning to hate speech detection. Available at SSRN **4902214**

[9] Vaidya, A., Arora, A., Joshi, A., Prabhakar, T.: Overview of the 2023 icon shared task on gendered abuse detection in indic languages. arXiv preprint **arXiv:2401.03677** (2024)

[10] Mandal, S., Sanand, S.: Strategies for language identification in code-mixed low resource languages. arXiv preprint **arXiv:1810.07156** (2018)

[11] Hua, Q., Qundong, S., Dingchao, J., Lei, G., Yanpeng, Z., Pengkang, L.: A character-level method for text classification. In: 2018 2nd IEEE Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC), pp. 402–406. IEEE, ??? (2018)

[12] Chauhan, D.S., Singh, G.V., Ekbal, A., Bhattacharyya, P.: Mhadig: A multilingual humor-aided multiparty dialogue generation in multimodal conversational

setting. Knowledge-Based Systems **278**, 110840 (2023)

[13] Hidayatullah, A.F., Qazi, A., Lai, D.T.C., Apong, R.A.: A systematic review on language identification of code-mixed text: techniques, data availability, challenges, and framework development. IEEE Access **10**, 122812–122831 (2022)

[14] Shekhar, S., Sharma, D.K., Beg, M.S.: Language identification framework in code-mixed social media text based on quantum lstm—the word belongs to which language? Modern Physics Letters B **34**(06), 2050086 (2020)

[15] Chaitanya, I., Madapakula, I., Gupta, S.K., Thara, S.: Word level language identification in code-mixed data using word embedding methods for indian languages. In: 2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI), pp. 1137–1141. IEEE, ??? (2018)

[16] Tonja, A.L., Yigezu, M.G., Kolesnikova, O., Tash, M.S., Sidorov, G., Gelbuk, A.: Transformer-based model for word level language identification in code-mixed kannada-english texts. arXiv preprint **arXiv:2211.14459** (2022)

[17] Jamatia, A., Das, A., Gambäck, B.: Deep learning-based language identification in english-hindi-bengali code-mixed social media corpora. Journal of Intelligent Systems **28**(3), 399–408 (2019)

[18] Dey, S., Thakur, S., Kandwal, A., Kumar, R., Dasgupta, S., Roy, P.P.: Bharatbhasanet-a unified framework to identify indian code mix languages. IEEE Access (2024)

[19] Mandal, S., Singh, A.K.: Language identification in code-mixed data using multichannel neural networks and context capture. arXiv preprint **arXiv:1808.07118** (2018)

[20] NLLB Team, Costa-jussa, M.R., Cross, J., et al.: No language left behind: Scaling human-centered machine translation. arXiv preprint **arXiv:2207.04672** (2022)

[21] Agarwal, O., Vyas, Y., Sharma, D., *et al.*: Limit: Less is more for multilingual language identification. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 14512–14528 (2023)

[22] Uchoi, E., Sambyo, K.: Code-switching scientific approaches using deep learning for english–kokborok language. National Academy Science Letters **49**, 1–9 (2026)